

GradeMate

Project Team

Amna Khan	21I-0345
Shanzay Alam	21I-2707
Ansab Alam	21I-2706

Session 2021-2025

Supervised by

Ms. Saira Qamar



Department of Artificial Intelligence

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.3	Modules	2
1.3.1	Collect Dataset	2
1.3.2	Model training	2
1.3.3	Solution Generation	3
1.3.4	Automated Grading	3
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Use-case Diagram	5
2.2	Functional Requirements	7
2.2.1	Module 1: Collecting Dataset	7
2.2.2	Module 2: Model Training	8
2.2.3	Module 3: Solution Generation	8
2.2.4	Module 4: Automated Grading	9
2.3	Non-Functional Requirements	9
2.3.1	Reliability	9
2.3.2	Usability	9
2.3.3	Performance	10
2.3.4	Security	10
2.4	Domain Model	11
3	System Overview	13
3.1	Architectural Design	13
3.2	Design Models	14
3.3	Data Design	22
3.3.1	Transformation of Information Domain into Data Structures	22
3.3.2	Storage of Major Data Entities	23
3.3.3	Processing of Major Data Entities	23
3.3.4	Organization of Data Entities	24
3.3.5	Databases and Data Storage Items	24

References

25

List of Figures

2.1	Use Case Diagram	5
2.2	Domain Model	11
3.1	The architectural design of the GradeMate system	13
3.2	Activity Diagram for Data collection Module	14
3.3	Activity Diagram for Model Training Module	15
3.4	Activity Diagram for Automated Grading Module	16
3.5	Activity Diagram for Solution Generation Module	17
3.6	Sequence Diagrams for OCR Conversion and Quiz Scanning	18
3.7	Sequence Diagrams for Solution Generation	19
3.8	Sequence Diagrams for Automated Grading	20
3.9	State Transition Diagram	21
3.10	Class Diagram	22

List of Tables

2.1	Use Case: Manage Handwritten Quizzes	6
2.2	Use Case: Manage Grading Criteria	6
2.3	Use Case: View Graded Quizzes	7
2.4	Use Case: Plagiarism Detection	7

Chapter 1

Introduction

The purpose of this project proposal is to provide a comprehensive overview of the development of GradeMate, which is an application designed to automate the grading process of handwritten quizzes written in C++ programming language. This document will outline the major aspects of the project including the objective, scope, and high-level goals of the system.

1.1 Problem Statement

The evaluation of handwritten programming quizzes in C++ can be a very time-consuming and error-prone process. Commonly practiced traditional manual grading methods can result in variability and vagueness, which can lead to unfair evaluation of student work. Additionally, teachers spend a high amount of time collecting, checking and grading each quiz submission, leaving less time for interactive teaching and focus for the students.[1]. One common issue is that a student attempts a quiz and provides a correct solution but uses a different approach than the one stated or expected by the teacher. Now the teacher, who has to manually check the quizzes of the entire class, struggles to assess this alternate solution.

1.2 Scope

GradeMate aims to develop a system that automates the grading of handwritten C++ programming quizzes. The primary functionality includes handwriting recognition, which will allow the conversion of handwritten C++ code into a digital format. Once digitized, the system will evaluate the code using predefined grading rubrics. Then the system will utilize AI-generated solutions, comparing multiple correct responses to the students' sub-

missions.

The system will incorporate secure cloud storage, ensuring that both quizzes and grading data are safely stored. Additionally, it will support batch processing, allowing several quizzes to be graded simultaneously, which is especially beneficial for larger class sizes.

Another key feature of GradeMate is plagiarism detection, which will help maintain academic integrity by identifying similarities between student submissions. The scope of the app will be limited to basic C++ concepts, excluding advanced topics like pointers and arrays, and will focus on scanning handwritten codes, grading automation, plagiarism detection, and cloud storage. The end goal is to produce a functional prototype that addresses the need for an efficient grading system for programming courses.

1.3 Modules

The proposed GradeMate application will be divided into several key modules, each responsible for handling specific functionalities. Below is an outline of the main modules and their respective features:

1.3.1 Collect Dataset

The user-friendly features offered will be focused on streamlining and improving the efficiency of the handwritten quiz collection, scanning, and digitization processes. These are some of this module's primary features:

1. OCR Conversion and Quiz Scanning
2. Template Matching
3. Text Cleaning and Formatting

1.3.2 Model training

The user will be able to utilize tools that help them optimize the training process and improve the AI model. Making sure the AI model will generate accurate quiz responses and understand the C++ programming language is the main objective.

1. Fine-tune GPT-Neo Model
2. Dataset Preprocessing and Cleaning
3. Training the Model with C++ solutions

1.3.3 Solution Generation

This module manages the quiz solution output, offering automated responses based on predefined templates and evaluating the output of the model. Features included are:

1. Automated Solution Generation
2. Solution Customization
3. Solution Validation

1.3.4 Automated Grading

This module is essential for the automatic grading of student quizzes and for identifying cases of plagiarism. Features provided to the user include:

1. Automated Grading System
2. Customizable Grading Rubrics
3. Plagiarism Detection

1.4 User Classes and Characteristics

With the primary goal of automating the grading of handwritten quizzes, the GradeMate system will serve various user classes within the educational system. The following user classes have been acknowledged along with the appropriate characteristics that they possess:

User class	Description
Instructor	Instructors will define grading criteria, approve rubrics, and oversee the grading process. The system will be used by them to make quizzes and track the completion of grading. When results from automated grading require manual confirmation, instructors may also come in. They will primarily use the system through a mobile application dashboard to oversee the process and analyze the outcomes.
Student	Students will also be the users of the system who will be submitting handwritten quizzes. These handwritten entries will be digitally converted by GradeMate, which will then automatically grade them. Multiple quizzes will be supported by the system, which is planned to be used by students using mobile applications. Additionally, students may interact with the system and get evaluations on their work.
Teaching Assistant (TA)	Teaching Assistants will be responsible for managing quiz submissions, reviewing the automated grading, and making any necessary manual adjustments. TAs will be the users of the system, often interacting with it to ensure grading accuracy and fairness. They may also generate quizzes using predefined templates and help configure grading rubrics.

Chapter 2

Project Requirements

2.1 Use-case Diagram

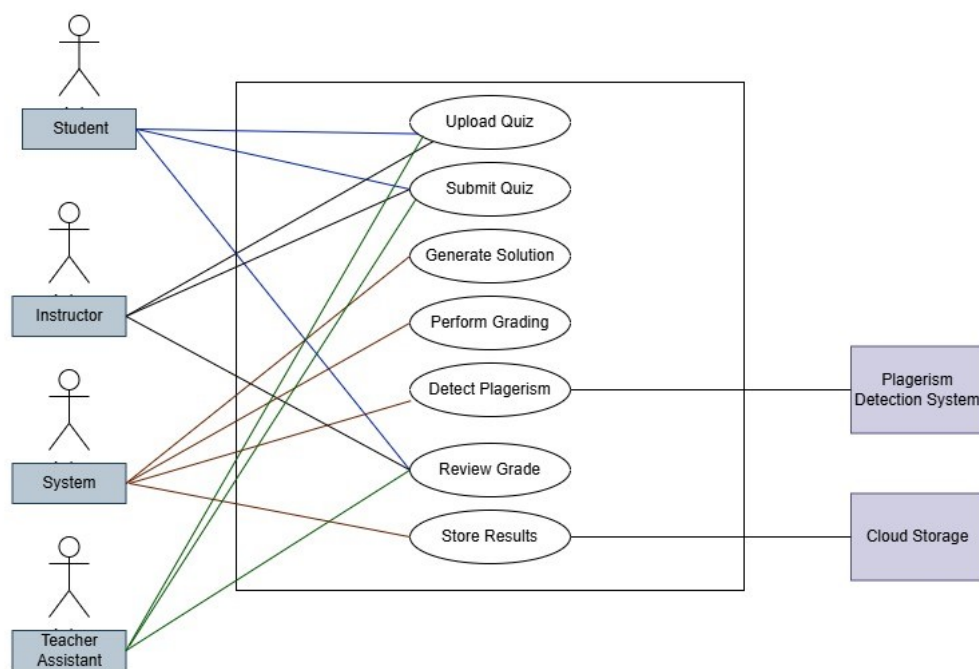


Figure 2.1: Use Case Diagram

Use Case	Manage Handwritten Quizzes
Actors	Teacher
Type	Primary
Organized By	Grading System
Description	The Teacher can upload handwritten quizzes to the system for grading, define the rubrics topics, and ensure student compliance.
Functional Requirements	• The user shall be able to upload handwritten quizzes. • The user shall define grading rubrics for each quiz. • The user shall ensure quizzes are stored securely in cloud storage.

Table 2.1: Use Case: Manage Handwritten Quizzes

Use Case	Manage Grading Criteria
Actors	Teacher
Type	Primary
Organized By	Grading System
Description	The Teacher can manage grading rubrics and criteria for evaluating student solutions.
Functional Requirements	• The Teacher shall create or edit grading rubrics. • The Teacher shall define weightage for each operation.

Table 2.2: Use Case: Manage Grading Criteria

Use Case	View Graded Quizzes
Actors	Student
Type	Primary
Organized By	Grading System
Description	Students can view their graded quizzes.
Functional Re-requirements	• The Student shall view scores and detailed feedback. • The Student shall access past submissions and grades for review.

Table 2.3: Use Case: View Graded Quizzes

Use Case	Plagiarism Detection
Actors	Teacher, Student,Any User
Type	Primary
Organized By	Grading System
Description	The system detects instances of plagiarism in students' submitted solutions to ensure academic integrity.
Functional Re-requirements	• The user shall be able to run plagiarism detection on submissions. • The system shall generate a report highlighting any detected plagiarism.

Table 2.4: Use Case: Plagiarism Detection

2.2 Functional Requirements

2.2.1 Module 1: Collecting Dataset

Following are the requirements for module 1:

1. FR1: Teachers and teaching assistants (TAs) will be able to upload handwritten quiz submissions in specified formats (JPEG and PNG) using the system.
2. FR2: The system shall automatically extract text from handwritten quiz submissions using Optical Character Recognition (OCR) tools

3. FR3: To increase model accuracy during training, the system should enable integration with external datasets (IAM, MNIST, InftyCBD-2 etc.).
4. FR4: The system will keep track of every quiz submission in the database, associating it to the student's individual ID and quiz number.

2.2.2 Module 2: Model Training

Following are the requirements for module 2:

1. FR1: The technology will enable the use of hybrid datasets, like EMNIST and quizzes submitted by students, to train machine learning models, including neural networks.
2. FR2: The system must enable saving of the trained model for further use and upgrades.
3. FR3: To enhance performance, the system should allow users (teachers or system administrators) to modify important model parameters including learning rate, batch size, and epochs.
4. FR4: Following each model training session, the system is required to produce performance metrics, such as accuracy, recall, and F1-score.
5. FR5: The system shall allow re-training of the model using new datasets or student submissions to ensure the model remains accurate and up-to-date.

2.2.3 Module 3: Solution Generation

Following are the requirements for module 3:

1. FR1: Teachers can create custom grading guidelines and rubrics for particular questions in the system.
2. FR2: The system shall compare the student's handwritten solution with the model-generated solution using a text similarity algorithm.
3. FR3: The system shall generate alternate proper solutions if students have solved the problem using different logic yet reached the correct answer.
4. FR4: Any differences between student and generated solutions should be recorded by the system for later analysis.

2.2.4 Module 4: Automated Grading

Following are the requirements for module 4:

1. FR1: Using the established rubrics, the system will automatically evaluate each quiz submission by comparing the responses provided by the students with the generated solutions.
2. FR2: The system will enable teachers to grade assignments manually, giving them the ability to modify or overturn grades that are generated automatically.
3. FR3: The system will link each grade to the appropriate quiz submission by storing student grades in the database.

2.3 Non-Functional Requirements

2.3.1 Reliability

1. REL-1: Under typical operating conditions, the GradeMate system must have a mean time between failures (MTBF) of at least a thousand hours.
2. REL-2: With a high accuracy rate, the system will identify grading errors or mismatches in student submissions.
3. REL-3: To prevent data loss in the case of a system failure, the system must enable teachers to access student information and grading history from the most recent checkpoint.
4. REL-4: Automated grading of quizzes shall have a less failure rate. Error logs will be maintained to record and notify any system failures

2.3.2 Usability

1. USE-1: The GradeMate system's user interface (UI) shall be intuitive and easy to use, for new users (teachers, TAs, and students).
2. USE-2: With just three clicks, the system should enable teachers to upload quizzes.
3. USE-3: Students will only need to click on their dashboard to view their graded quizzes.
4. USE-4: The system will offer error alerts and help for typical user mistakes, including uploading the wrong file format for a quiz.

2.3.3 Performance

1. PER-1: The system shall process the upload of a handwritten quiz in under 60 seconds with faster internet connection.
2. PER-2: For quizzes with 20 submissions or more, automatically graded results must be accessible in 10 minutes.
3. PER-3: During peak hours, the system can accommodate up to 100 students and 50 teachers using it concurrently without experiencing any performance reduction.
4. PER-4: For the best possible user experience, the GradeMate system must load each page or activity in less than two seconds.

2.3.4 Security

1. SEC-1: The system must guarantee that encryption standards are applied to all quiz data, student submissions, and grade.
2. SEC-2: The users should add credentials to enforce user authentication for instructors and teaching assistants.
3. SEC-3: To guard against unauthorized access or modifications, the system must record and keep track of every activity made by users (teachers, TAs, and students). These logs must be kept for a period of 90 days.
4. SEC-4: Access control will be put into place to make sure that only individuals with permission can read and edit tests, grades, or student information. After three unsuccessful unauthorized access attempts, the user account will be locked and alerts will be sent out.

2.4 Domain Model

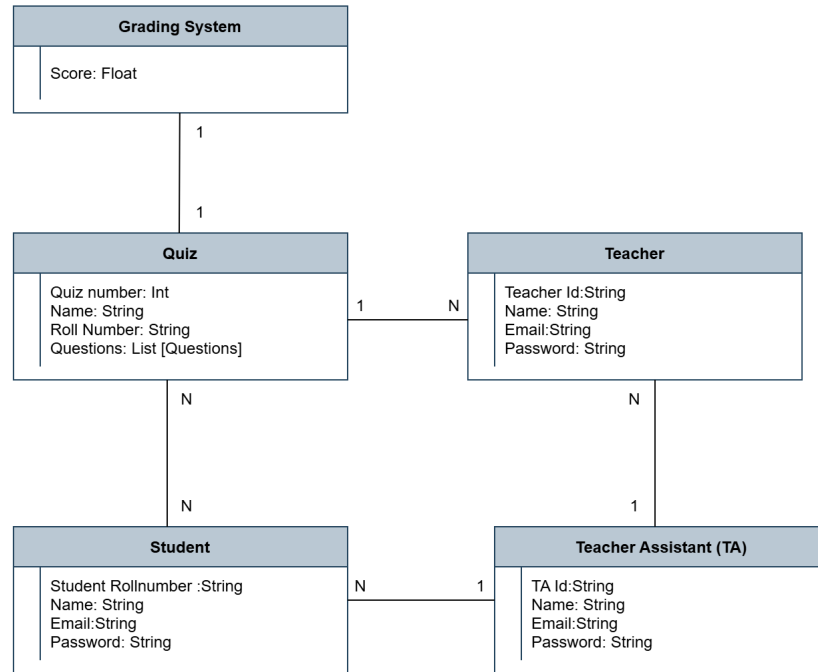


Figure 2.2: Domain Model

Chapter 3

System Overview

3.1 Architectural Design

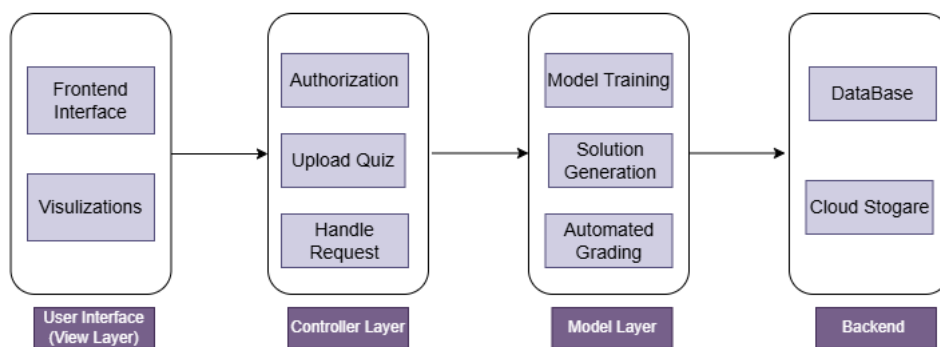


Figure 3.1: The architectural design of the GradeMate system

3.2 Design Models

Activity Diagrams

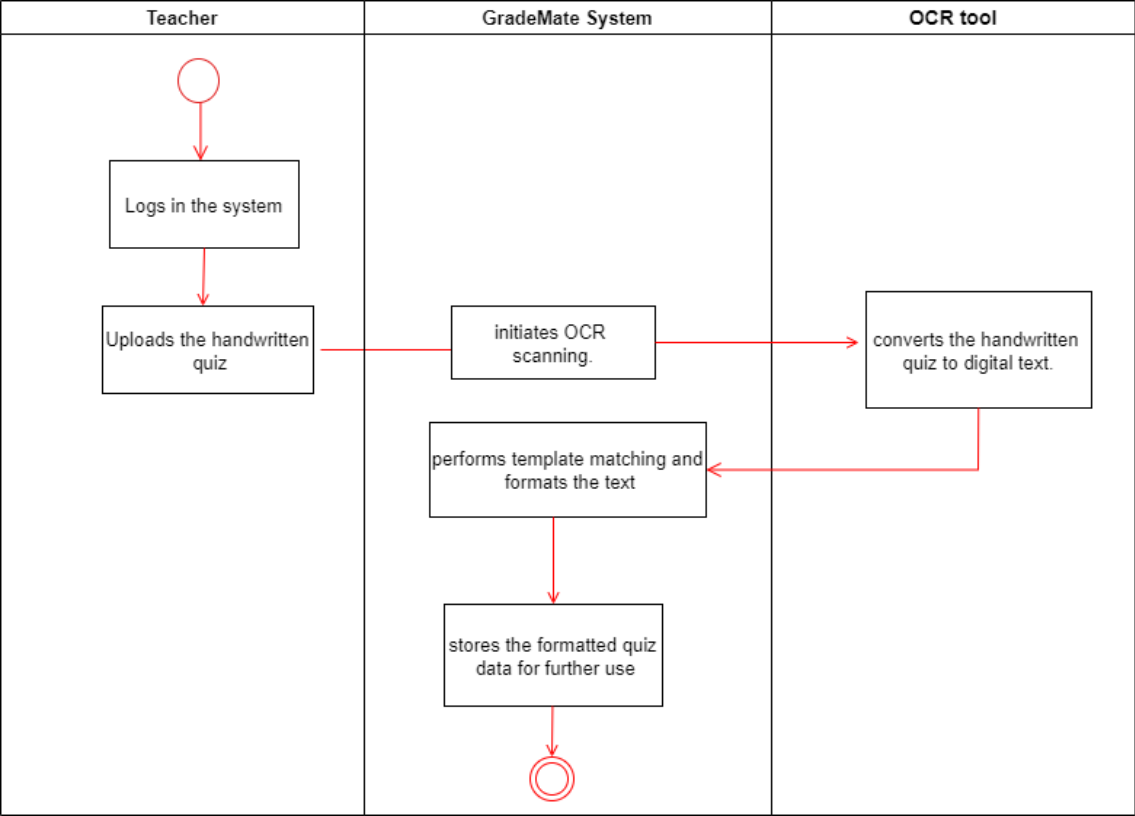


Figure 3.2: Activity Diagram for Data collection Module

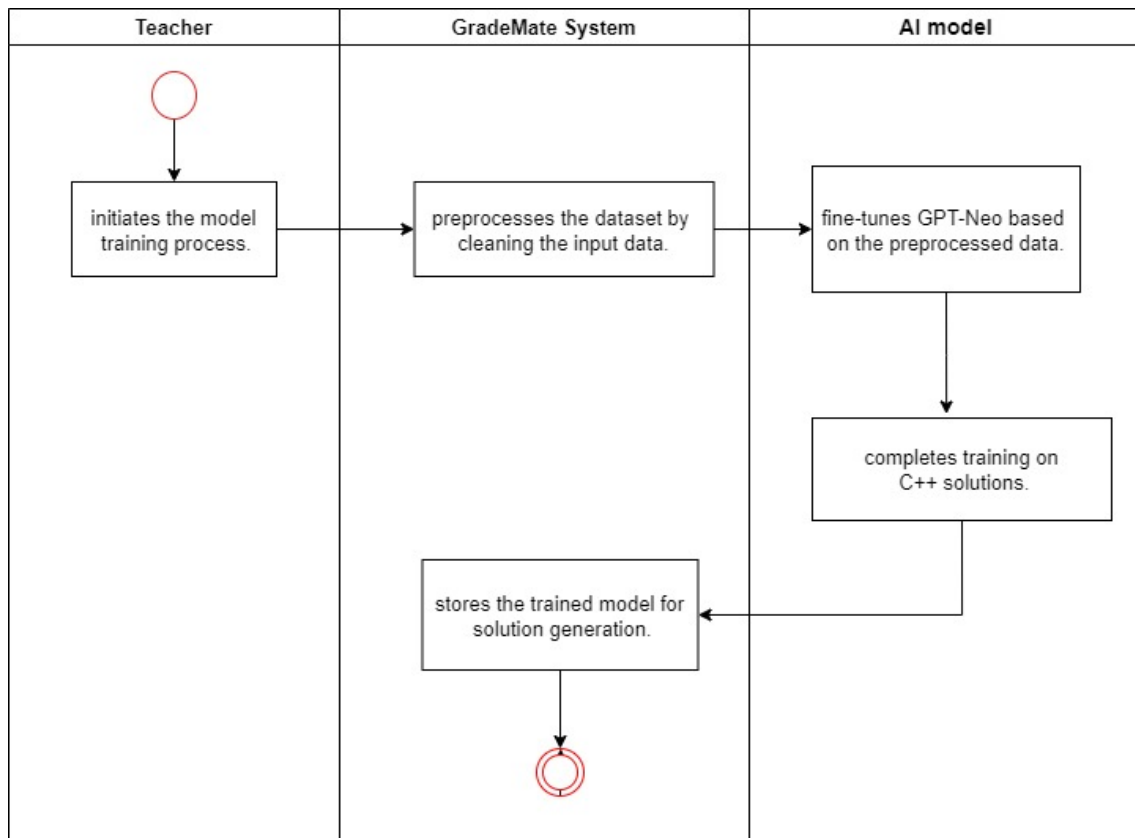


Figure 3.3: Activity Diagram for Model Training Module

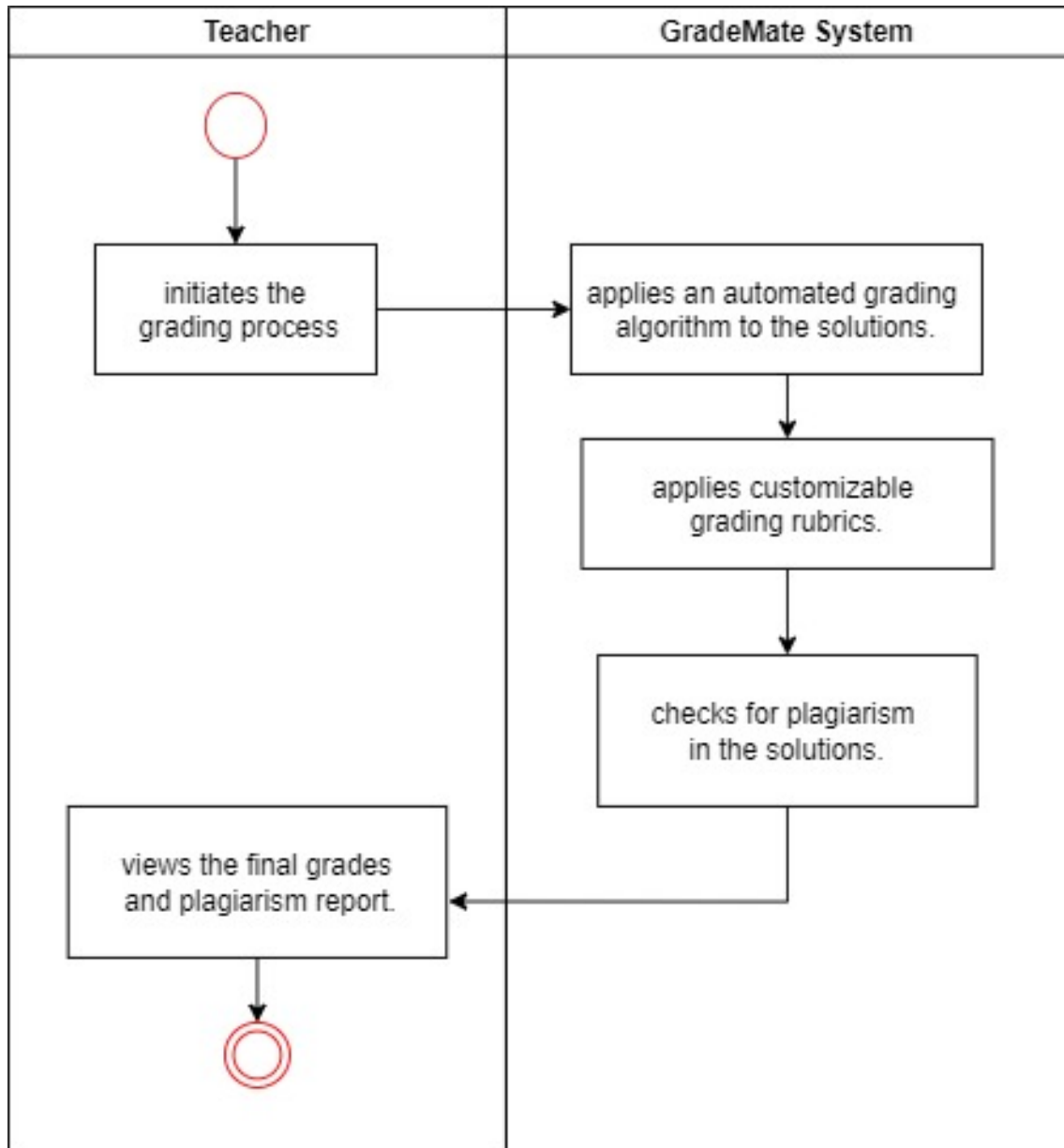


Figure 3.4: Activity Diagram for Automated Grading Module

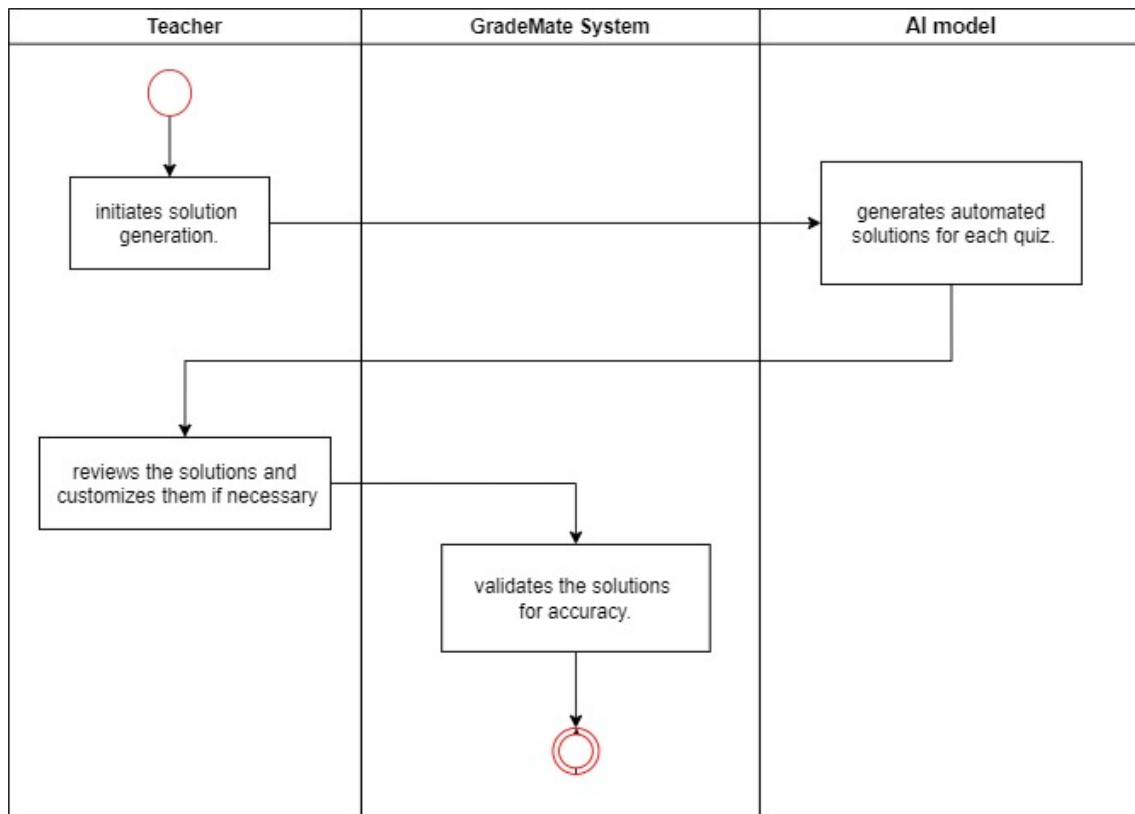


Figure 3.5: Activity Diagram for Solution Generation Module

Sequence Diagrams

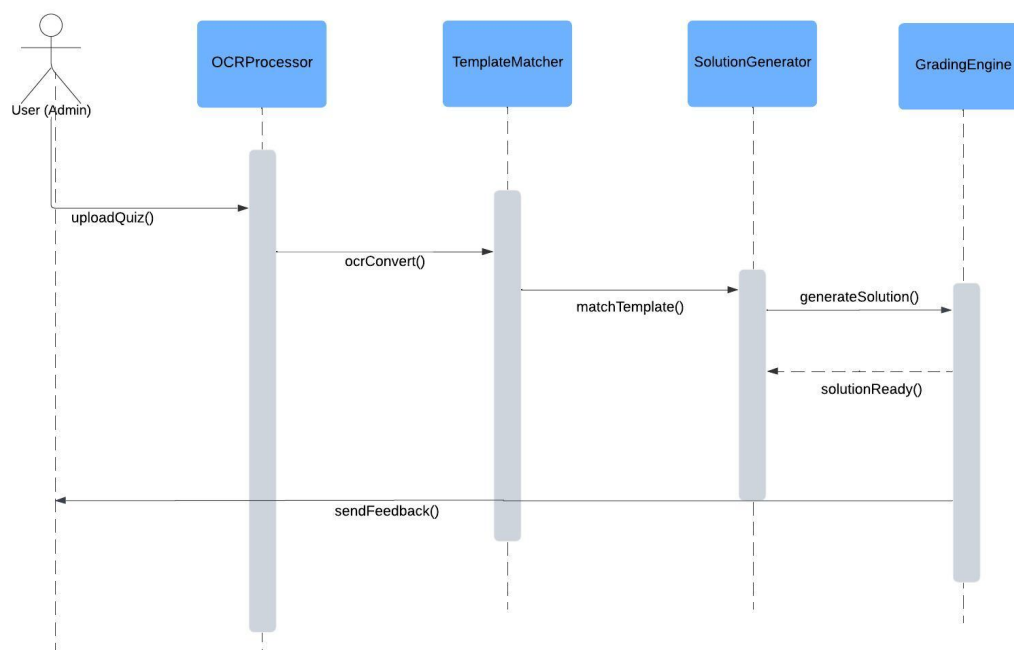


Figure 3.6: Sequence Diagrams for OCR Conversion and Quiz Scanning

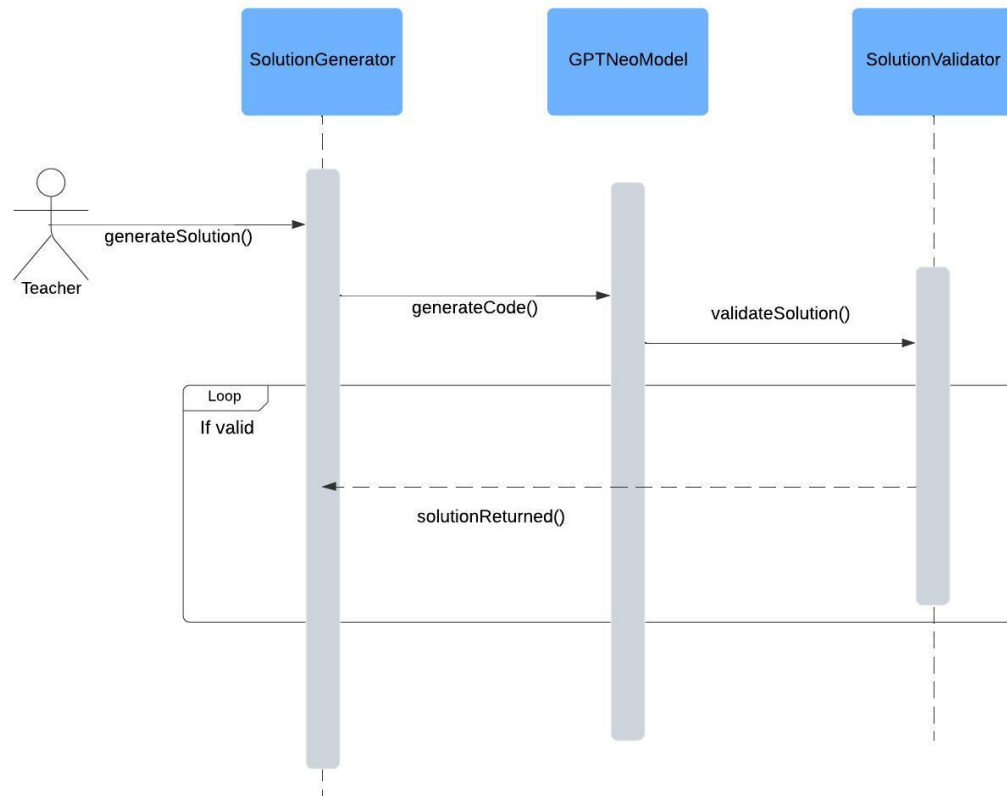


Figure 3.7: Sequence Diagrams for Solution Generation

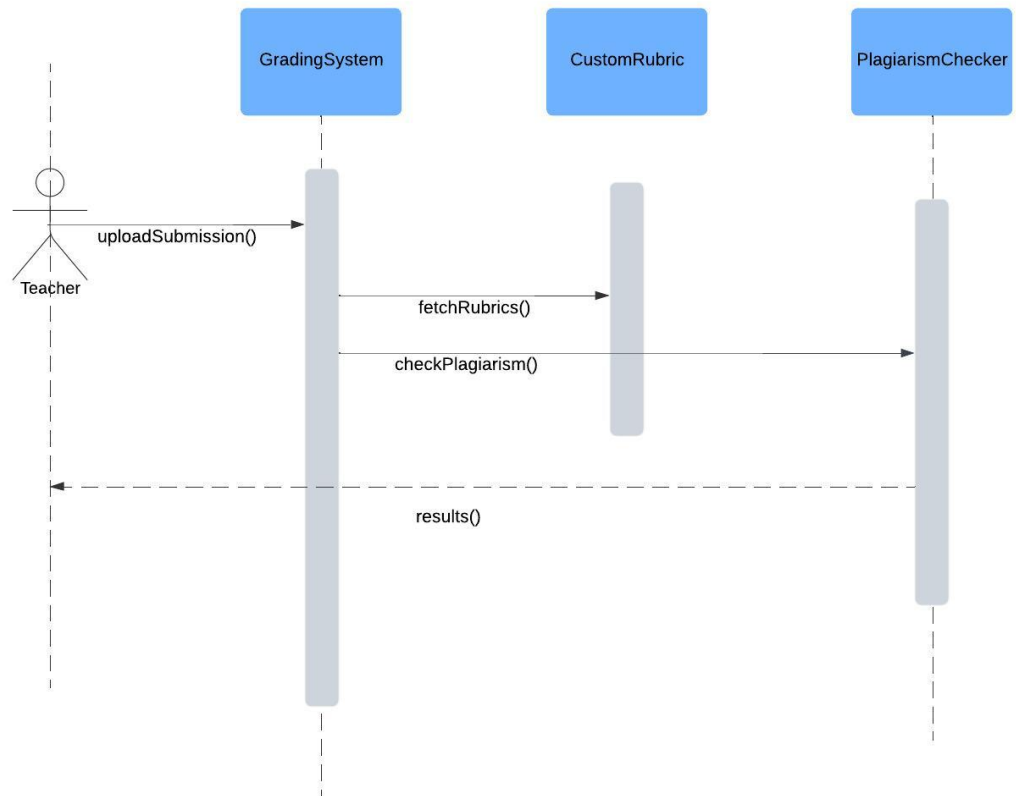


Figure 3.8: Sequence Diagrams for Automated Grading

State Transition Diagram

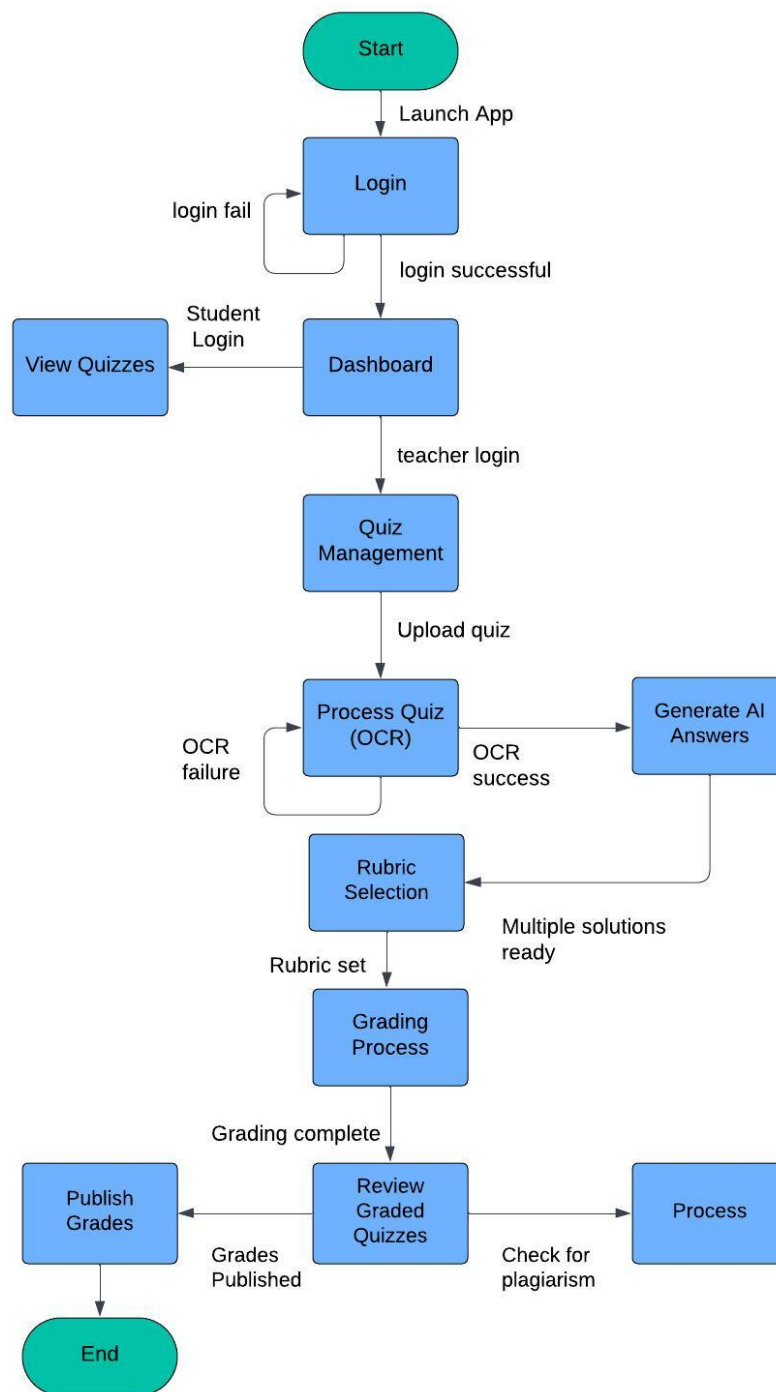


Figure 3.9: State Transition Diagram

Class Diagram

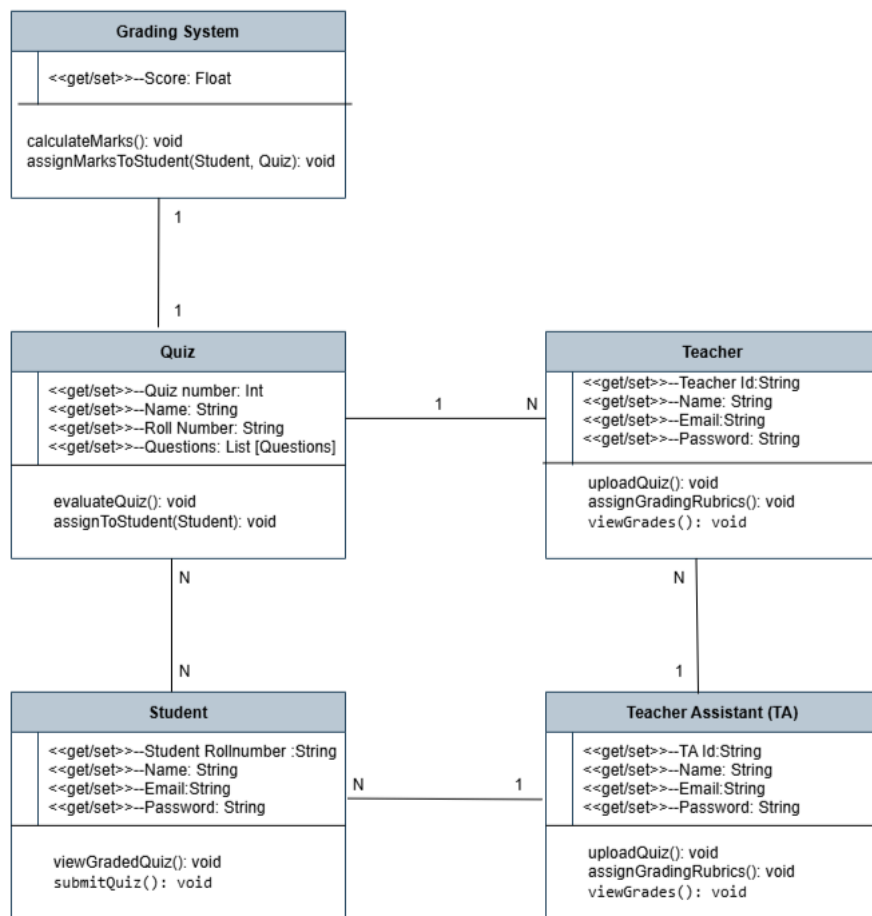


Figure 3.10: Class Diagram

3.3 Data Design

3.3.1 Transformation of Information Domain into Data Structures

Structured, semi-structured, and unstructured data (such as handwritten quizzes) are all handled by the system. The process of converting information into data structures is as follows:

1. Quizzes: The primary data entity is quizzes, each of which has different attributes: . It is linked to student submissions and grades.

2. Users: Multiple user roles, such as students, teachers, and teaching assistants (TAs), are supported by the system. A user's name, email address, user type, credentials, and user ID are among their attributes.
3. HandWritten Quiz Data: To extract text from student quiz images, OCR (Optical Character Recognition) is used. These images are stored and linked to the corresponding quizzes and students.
4. Grades: Each quiz attempt is graded, and the grades are stored in structured format with attributes like marks, and quizNumber.

3.3.2 Storage of Major Data Entities

1. Relational Databases: Relational databases like MySQL are used to store structured data, such as user information, quiz results, and grades. The following tables are widely used:
 - Users Table: Stores user data, differentiating between students, teachers, and TAs.
 - Quizzes Table: Holds details about every quiz.
 - Grades Table: Using foreign keys to link students and quizzes, this table holds the grades for each attempt at a quiz.
2. Handwritten Quiz Images: Handwritten quizzes are kept in cloud storage systems. These files are linked to quizzes and students in the relational database.
3. Plagiarism Data: The system stores data from plagiarism detection processes. The Plagiarism Results Table holds attributes such as plagiarismScore, and comparison-Details

3.3.3 Processing of Major Data Entities

- OCR Processing: OCR tools are used to extract text, which is then compared against a set of model solutions.
- Automated Grading: The system compares students' responses to model solutions using a combination of machine learning and pre-established grading rubrics. Accuracy, completeness, and compliance to the question structure are taken into consideration while awarding grades.
- Plagiarism Detection: To identify possible plagiarism, programs such as MOSS will compare student submissions to both previous and current data.

3.3.4 Organization of Data Entities

1. User-Based Data Organization: Data is structured based on the role of the user:
2. Quiz Data Organization: Each quiz is linked to its respective students, with relationships.

3.3.5 Databases and Data Storage Items

1. Relational Database (MySQL) Stores structured data such as quiz information, grades, user details, and plagiarism results.
2. Cloud Storage Stores unstructured data such as images of handwritten quizzes

Bibliography

- [1] Claus Aranha Dominik Sobania, Christin Seifert. Artificial intelligence in education: Learning and teaching in the digital world. *Automated grading: Current status and outlook for the future*, 2022.